

Git Merge Conflict - Local System Practice Assignment

Git Merge Conflict - Local System Practice Assignment

Objective

Practice creating and resolving a merge conflict in the local system.

In this assignment, you will:

```
create local project
initialize Git
rename branch to main if needed
create a branch
change same line in two branches
merge branches
resolve conflict
complete merge
```

Important Caution: **master** vs **main**

After running:

```
git init
```

some systems may create the default branch as:

```
master
```

But for this assignment, we will use:

```
main
```

So after `git init`, always run:

```
git branch -M main
```

This renames the current branch to `main`.

Step 1: Create a Folder

Create a folder on Desktop named:

```
merge-conflict-practice
```

Open vs open.

Click:

```
Open Folder
```

Select the folder:

```
merge-conflict-practice
```

Step 2: Create `index.html`

Create a file named:

```
index.html
```

Add this code:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Merge Conflict Practice</title>
  </head>
  <body>
    <h1>Welcome to Git Practice</h1>
    <p>This file is created on the main branch.</p>
  </body>
</html>
```

Save the file.

Step 3: Initialize Git

Open terminal inside Antigravity and run:

```
git init
git branch -M main
git status
git add .
git commit -m "initial commit"
```

Check current branch:

```
git branch
```

You should see:

```
* main
```

Step 4: Create a New Branch

Create and switch to a new branch:

```
git switch -c heading-change
```

Open `index.html`.

Change this line:

```
<h1>Welcome to Git Practice</h1>
```

To:

```
<h1>Hello from heading-change branch</h1>
```

Save the file.

Stage and commit:

```
git status
git add .
git commit -m "changed heading in branch"
```

Step 5: Change Same Line on `main`

Switch back to `main`:

```
git switch main
```

Open `index.html`.

Change the same heading line to:

```
<h1>Hello from main branch</h1>
```

Save the file.

Stage and commit:

```
git status
git add .
git commit -m "changed heading in main"
```

Step 6: Merge and Create Conflict

Now merge `heading-change` into `main`:

```
git merge heading-change
```

You should see a conflict message similar to:

```
CONFLICT (content): Merge conflict in index.html
Automatic merge failed; fix conflicts and then commit the result.
```

Step 7: Open the Conflict File

Open `index.html`.

You will see something like this:

```
<<<<<< HEAD
<h1>Hello from main branch</h1>
=====
<h1>Hello from heading-change branch</h1>
>>>>>> heading-change
```

Meaning:

```
HEAD = current branch content
===== = separator
heading-change = incoming branch content
```

Since we are currently on `main`:

```
HEAD means main branch content.
```

Step 8: Resolve the Conflict

Remove the conflict markers:

```
<<<<<< HEAD
=====
>>>>>> heading-change
```

Choose the final heading.

Use this final code:

```
<h1>Hello from main and heading-change branch</h1>
```

Your final `index.html` should look normal again:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Merge Conflict Practice</title>
  </head>
  <body>
    <h1>Hello from main and heading-change branch</h1>
    <p>This file is created on the main branch.</p>
  </body>
</html>
```

Save the file.

Step 9: Complete the Merge

After resolving the conflict, run:

```
git status
git add .
git commit -m "resolved heading conflict"
```

Important:

Editing the file is not enough.
After fixing the conflict, you must run git add and git commit.

Step 10: View Commit History

Run:

```
git log --oneline
```

You should see commits for:

```
initial commit
changed heading in branch
changed heading in main
resolved heading conflict
```